

Code-Layout, Readability and Reusability

Date	1 October 2026
Team ID	SWTID-2026-2533
Project Name	EduGenie Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document reviews the EduGenie codebase for structure, readability and reuse: how the files are organized, how clearly the code reads, and which parts can be reused in later features.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing is used throughout the code	Yes	4 spaces in every Python file (PEP 8) and 2 spaces in the HTML, CSS and JavaScript.
2	Proper File Structure	Files and folders are logically organized	Yes	One module per feature (qna.py, quiz_module.py, summary_module.py, learning_path.py, explanation_module.py), main.py for routes, and templates/ and static/ for the page.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names such as topic, quiz, MODEL_NAME and cleaned JSON fields say what they hold.
4	Function / Method Names	Functions are descriptively named	Yes	Action-verb names: answer_question_with_gemini, generate_quiz, summarize_text, explain_topic, get_learning_recommendations.
5	Code Comments	Inline and block comments explain the logic	Yes	Each module opens with a docstring, and key steps (JSON fence cleaning, lazy model loading, Gemini fallback) carry short comments.
6	Modular Design	Code is split into reusable functions and modules	Yes	AI calls live in the feature modules; main.py only handles routing and error responses.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Every Gemini call goes through one generate() helper, and one run() helper handles errors for all endpoints.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Empty input returns HTTP 400; AI and network errors are caught and returned as clear messages; quiz output is validated before use.
9	Automated Tests	Unit tests cover the main logic	No	Features were tested manually through the web page. Automated tests for the quiz parser and the API endpoints are planned.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Gemini Client Helper (gemini_client.py)	Python / Google GenAI SDK	Used by Q&A, Quiz, Summary and Learning Path, and as the fallback for Explanation. It holds the API key check, the model name and the JSON option in one place.	High
2	Request and Error Helpers (run(), error(), read_field() in main.py)	Python / FastAPI	Shared by all five endpoints to read input, run AI calls off the main thread and return uniform JSON errors.	High
3	Markdown Renderer (renderMarkdown() in index.html)	JavaScript	Shared by the Answer, Explanation, Summary and Learning Path results to show headings, bold text and bullet lists safely.	High
4	Quiz JSON Cleaner and Validator (clean_json_block(), _normalize())	Python / Regular Expressions	Used in the quiz pipeline; can be reused by any feature that expects structured JSON from Gemini.	Medium
5	Local Model Loader (explanation_module.py)	Python / Transformers, PyTorch	Loads LaMini-Flan-T5 once, on first use, behind a thread lock, and shares it across every explain request.	Medium
6	Page Stylesheet (static/style.css)	CSS3	Colour and spacing variables are reused by the form, results, quiz and error styles, with one mobile breakpoint.	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Each feature has its own module, routing is separate from AI logic, and the page assets sit in templates/ and static/.
Readability	5	Short functions with descriptive names and docstrings; no deeply nested logic.
Reusability	4	The Gemini helper, error helpers and Markdown renderer are shared widely. The local model loader is specific to explanations.
Documentation / Comments	4	Docstrings and key comments are present, and the setup steps are written down in the Coding & Solution document. Automated tests are still to be added.
Overall Score	4.5	Clean, modular and easy to extend. The next improvements are automated tests and a progress-tracking module.